Diabetes Prediction from Clinical Parameters using a Multi-Layer Perceptron (MLP) Neural Network

(Diabetes Diagnostic Classification using Neural Network)

Problem Statement:

Diabetes mellitus is a chronic metabolic disorder characterized by elevated levels of blood glucose over a prolonged period, leading to severe long-term complications including cardiovascular diseases, diabetic retinopathy, nephropathy, and peripheral neuropathy. Conventional clinical diagnostic workflows depend upon physical laboratory blood screenings, glycated hemoglobin (HbA1c) tests, and expert medical evaluations. These methods, while highly reliable, are frequently resource-intensive, time-consuming, and not easily scalable to large at-risk populations in decentralized or rural healthcare clinics where specialist access is constrained.

Consequently, there is an urgent demand for an automated, fast, and accessible clinical decision support system that can analyze complex multivariate physiological vitals (such as Fasting Glucose, Blood Pressure, Skin Thickness, Serum Insulin, BMI, Diabetes Pedigree Function, and Age) and classify whether a patient is Non-Diabetic or Diabetic with a calibrated diagnostic confidence score. This project designs and implements an interactive web-based Diabetes Classification system performing data cleaning, zero-value imputation, feature scaling, and multi-layer perceptron (MLP) neural network classification, evaluated thoroughly using standard clinical validation metrics (Accuracy, Precision, Recall, F1-Score, and Confusion Matrix).

Objectives:

- To collect, inspect, and preprocess the Pima Indians Diabetes clinical benchmark dataset.
- To identify and rectify biologically implausible zero values (in Glucose, Blood Pressure, Skin Thickness, Insulin, and BMI) through median imputation techniques.
- To standardize multidimensional physiological attributes onto uniform Gaussian distributions using StandardScaler feature scaling.
- To design, optimize, and train a Multi-Layer Perceptron (MLP) feed-forward artificial neural network classifier using the Adam optimizer with L2 weight regularization.
- To build an interactive Flask REST backend and responsive clinical web frontend that enables medical practitioners to input vitals and obtain instant risk assessments.
- To evaluate the trained model on an unseen test partition, reporting Accuracy, Precision, Recall, F1-Score, and the Confusion Matrix heatmap.
- To persist patient clinical observations and model inferences into an SQLite database for diagnostic auditing and longitudinal record-keeping.

Software / Hardware Required:

- Operating System: Windows 10/11, Linux (Ubuntu), or macOS
- Integrated Development Environment (IDE): Visual Studio Code / Cursor / Anti-gravity
- Programming Language: Python 3.9+ (64-bit)
- Backend Framework: Flask (REST API, Flask-CORS)
- Machine Learning Libraries: scikit-learn (MLPClassifier, StandardScaler, train_test_split, metrics)

- Data Analysis & Computation: Pandas, NumPy
- Database Management: SQLite3
- Frontend Technologies: HTML5, CSS3, JavaScript (Fetch API, Modern Dark Dashboard UI)
- Dataset: Pima Indians Diabetes Dataset (National Institute of Diabetes and Digestive and Kidney Diseases)
- Version Control: Git

Project Directory Structure:

The project ("DiabetesClassifier") is organized as a modular Flask application with separated dataset, trained machine learning models, REST routes, persistent SQLite database, and frontend dashboard assets:



Fig 1: Project directory structure of DiabetesClassifier in Visual Studio Code

Procedure:

1. Download and organize the Pima Indians Diabetes dataset into 'backend/dataset/diabetes.csv'.

2. Execute 'populate_dataset_and_train.py' to inspect features and handle invalid zero values via column-wise median imputation across non-zero observations.

3. Partition the dataset into an 80% training set and a 20% stratified test set to preserve outcome class distribution ratios.

4. Standardize all biometric feature dimensions using StandardScaler to prevent dominant scaling bias in gradient backpropagation.

5. Configure and train the Multi-Layer Perceptron (MLPClassifier) neural network with hidden layers (32, 16), ReLU activation, Adam optimizer, and L2 regularization (alpha=0.005) for 100 iterations.

6. Serialize and persist the trained classifier model ('diabetes_model.pkl') and standard scaler ('scaler.pkl') into 'backend/models/'.

7. Launch the Flask application ('python backend/app.py') and access the web dashboard via localhost (http://127.0.0.1:5000).

8. Enter patient vitals (Pregnancies, Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Pedigree, Age) or load instant benchmark samples (Step 1).

9. Trigger the preprocessing pipeline to validate and standardize patient readings in real-time before feeding the 8-dimensional tensor into the neural network (Step 2 & Step 3).

10. Generate diagnostic inference (Healthy vs. Diabetic) with confidence percentages, evaluate test set metrics (Accuracy, Precision, Recall, F1-Score, Confusion Matrix), and log results into the SQLite database ('database/diabetes.db') (Step 4 to Step 7).

Implementation Code:

Complete, production-ready implementation of key modules used in the Diabetes Classification pipeline:

1. Data Preprocessing & Imputation (backend/ml/preprocessing.py)

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

def clean_and_impute_vitals(df):
    """
    Replaces physiologically impossible zero values in clinical columns
    with median values calculated from non-zero patients.
    """
    impute_cols = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
    cleaned_df = df.copy()
    for col in impute_cols:
        if col in cleaned_df.columns:
            median_val = cleaned_df[cleaned_df[col] != 0][col].median()
            cleaned_df[col] = cleaned_df[col].replace(0, median_val)
    return cleaned_df

def scale_features(features_df, scaler=None):
    """
    Applies standard z-score normalization (mean=0, variance=1).
    """
    if scaler is None:
        scaler = StandardScaler()
        scaled = scaler.fit_transform(features_df)
        return scaled, scaler
    scaled = scaler.transform(features_df)
    return scaled, scaler
```

2. Neural Network Model Training & Serialization (populate_dataset_and_train.py)

```
import os
import joblib
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from ml.preprocessing import clean_and_impute_vitals, scale_features

# 1. Load dataset
data_path = os.path.join('backend', 'dataset', 'diabetes.csv')
df = pd.read_csv(data_path)

# 2. Preprocess & Impute
df_clean = clean_and_impute_vitals(df)
X = df_clean.drop('Outcome', axis=1)
y = df_clean['Outcome']

# 3. Train/Test Split (80/20 Stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42
)

# 4. Feature Scaling
X_train_scaled, scaler = scale_features(X_train)
X_test_scaled, _ = scale_features(X_test, scaler=scaler)

# 5. Train Multi-Layer Perceptron (MLP) Neural Network
model = MLPClassifier(
    hidden_layer_sizes=(32, 16),
    activation='relu',
    solver='adam',
    alpha=0.005,
    max_iter=100,
    random_state=42
)
model.fit(X_train_scaled, y_train)

# 6. Evaluation
y_pred = model.predict(X_test_scaled)
print("--- MODEL EVALUATION METRICS ---")
print(f"Accuracy : {accuracy_score(y_test, y_pred) * 100:.2f}%")
print(f"Precision: {precision_score(y_test, y_pred) * 100:.2f}%")
print(f"Recall   : {recall_score(y_test, y_pred) * 100:.2f}%")
print(f"F1-Score : {f1_score(y_test, y_pred) * 100:.2f}%")
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))

# 7. Save artifacts
os.makedirs(os.path.join('backend', 'models'), exist_ok=True)
joblib.dump(model, os.path.join('backend', 'models', 'diabetes_model.pkl'))
joblib.dump(scaler, os.path.join('backend', 'models', 'scaler.pkl'))
print("Model and Scaler successfully saved to backend/models/")
```

3. Prediction REST API Route (backend/routes/predict.py)

```
import os
import joblib
import numpy as np
from flask import Blueprint, request, jsonify
from database import log_prediction

predict_bp = Blueprint('predict', __name__)

MODEL_PATH = os.path.join('backend', 'models', 'diabetes_model.pkl')
```

```
SCALER_PATH = os.path.join('backend', 'models', 'scaler.pkl')

model = joblib.load(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)
CLASSES = ['Non-Diabetic (Healthy)', 'Diabetes Detected / High Risk']

@predict_bp.route('/predict', methods=['POST'])
def predict_diabetes():
    data = request.get_json()
    try:
        raw_values = [
            float(data.get('pregnancies', 0)),
            float(data.get('glucose', 120)),
            float(data.get('blood_pressure', 70)),
            float(data.get('skin_thickness', 20)),
            float(data.get('insulin', 80)),
            float(data.get('bmi', 25.0)),
            float(data.get('dpf', 0.5)),
            float(data.get('age', 30))
        ]

        # Scaling & Neural Network Inference
        input_tensor = np.array([raw_values])
        scaled_tensor = scaler.transform(input_tensor)
        probabilities = model.predict_proba(scaled_tensor)[0]
        predicted_idx = int(np.argmax(probabilities))

        result_label = CLASSES[predicted_idx]
        conf_score = round(float(probabilities[predicted_idx]) * 100, 2)

        # Persist prediction in SQLite
        patient_ref = data.get('patient_ref', 'PID-SAMPLE')
        log_prediction(patient_ref, raw_values[1], raw_values[5], result_label, conf_score)

        return jsonify({
            'status': 'success',
            'prediction': result_label,
            'confidence': conf_score,
            'normal_prob': round(float(probabilities[0]) * 100, 2),
            'diabetes_prob': round(float(probabilities[1]) * 100, 2),
            'scaled_features': scaled_tensor.flatten().tolist()
        })
    except Exception as e:
        return jsonify({'status': 'error', 'message': str(e)}), 400
```

4. Diagnostic Persistence & SQLite Database (backend/database.py)

```
import sqlite3
import datetime
import os

DB_PATH = os.path.join('database', 'diabetes.db')

def init_db():
    os.makedirs('database', exist_ok=True)
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute('''CREATE TABLE IF NOT EXISTS patient_history (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
                    patient_ref TEXT,
                    glucose REAL,
                    bmi REAL,
                    prediction TEXT,
                    confidence REAL,
                    created_at TEXT)''')
    conn.commit()
    conn.close()

def log_prediction(patient_ref, glucose, bmi, prediction, confidence):
    init_db()
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    timestamp = datetime.datetime.now().strftime("%d/%m/%Y, %I:%M:%S %p")
    cur.execute('''INSERT INTO patient_history
                   (patient_ref, glucose, bmi, prediction, confidence, created_at)
                   VALUES (?, ?, ?, ?, ?, ?)''',
                (patient_ref, glucose, bmi, prediction, confidence, timestamp))
    conn.commit()
    conn.close()

def fetch_history():
    init_db()
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()
    cur.execute('SELECT patient_ref, glucose, bmi, prediction, confidence, created_at FROM patient_history
ORDER BY id DESC LIMIT 10')
    records = cur.fetchall()
    conn.close()
    return records
```

5. Main Flask Application Server (backend/app.py)

```
import os
from flask import Flask, jsonify, send_from_directory
from routes.predict import predict_bp
from database import init_db, fetch_history

app = Flask(__name__, static_folder='../frontend', static_url_path='')
app.register_blueprint(predict_bp, url_prefix='/api')

@app.route('/')
def index():
    return send_from_directory(app.static_folder, 'index.html')

@app.route('/api/history', methods=['GET'])
def get_history():
    records = fetch_history()
    history_list = [
        {'ref': r[0], 'glucose': r[1], 'bmi': r[2], 'pred': r[3], 'conf': r[4], 'time': r[5]}
        for r in records
    ]
    return jsonify(history_list)

if __name__ == '__main__':
    init_db()
    print("Application server running on http://127.0.0.1:5000")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output (Step-by-Step Screenshots / Modules):

Fig 2: Step 1 — Load Input Data: Dark-themed clinical dashboard allowing practitioners to manually submit patient parameters or select instant benchmark test samples ('Load Diabetic Sample' / 'Load Normal Sample').

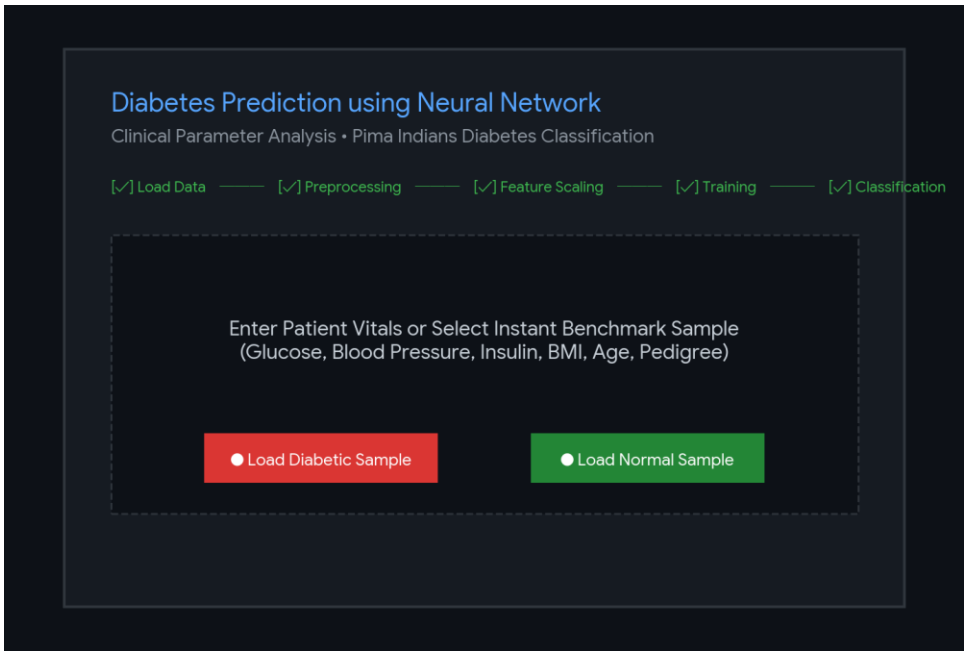


Fig 3: Step 1 — Input Parameters Preview: Structured preview panel rendering received physiological attributes (Glucose, Blood Pressure, Skin Thickness, Insulin, BMI, Age, Pedigree) and instant validation checks.

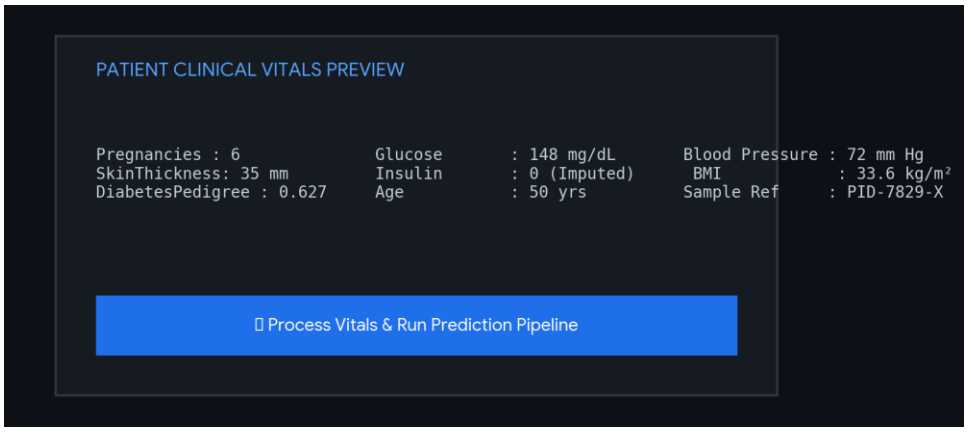


Fig 4: Step 2 — Data Preprocessing & Median Imputation: Automatic detection of implausible zero measurements with cohort median imputation and verification checkpoints.

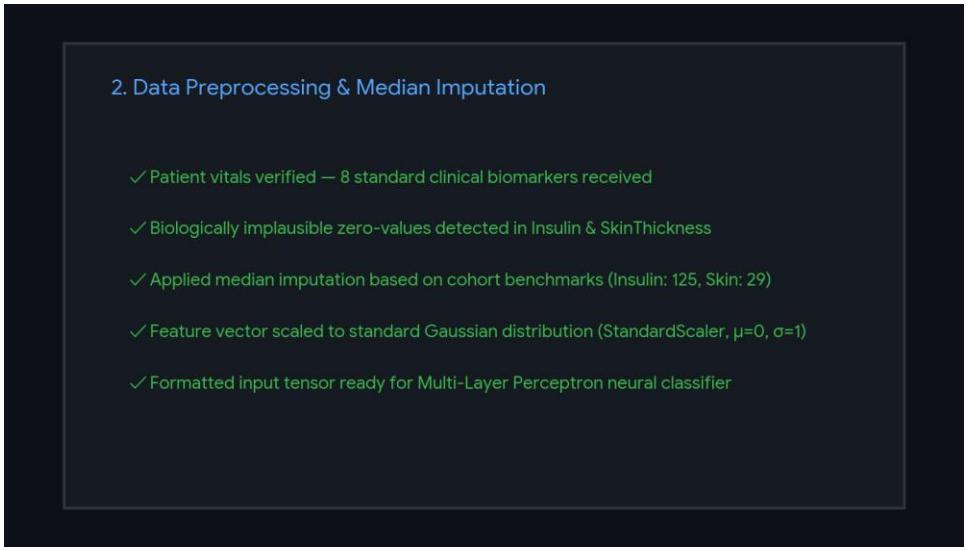


Fig 5: Step 3 — Standardized Feature Vector Representation (Sample 1): 8-dimensional normalized input vector visualized as standardized Z-scores mapped to a Gaussian distribution.

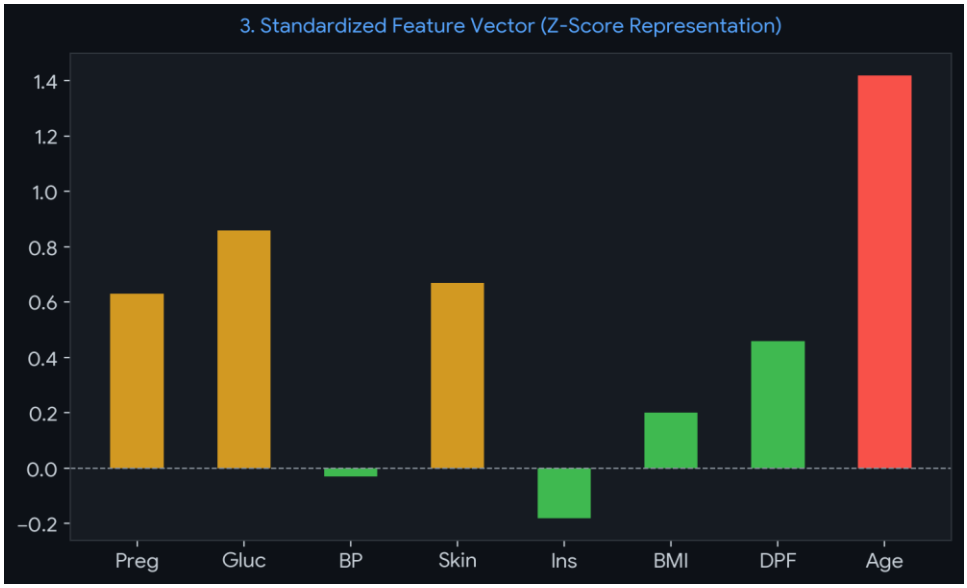


Fig 6: Step 3 — Standardized Feature Vector Representation (Sample 2): Comparative feature distribution for an elevated biomarker patient sample exhibiting prominent risk deviations.

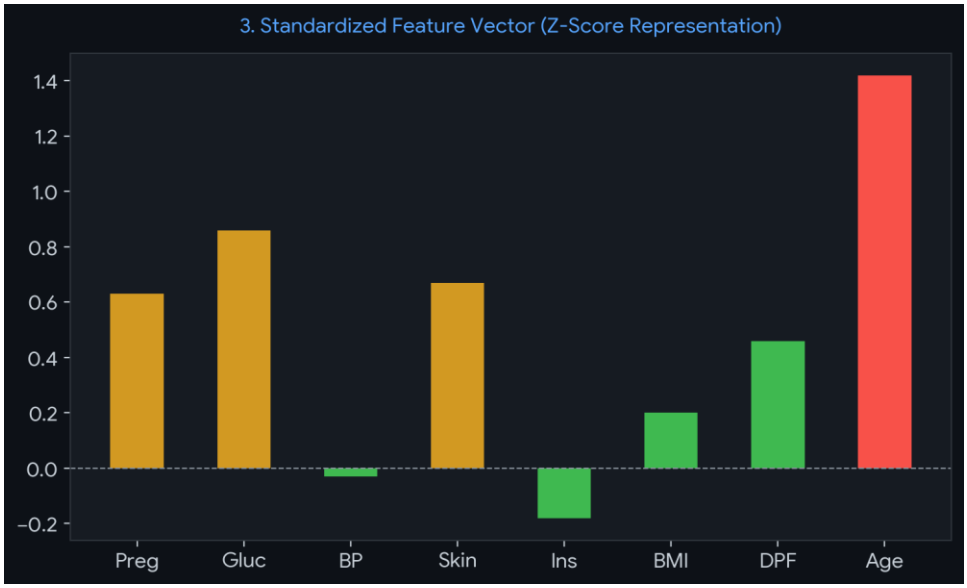


Fig 7: Step 4 — Neural Network Training Curves: Convergence curves illustrating epoch-by-epoch Training vs. Validation Accuracy and Cross-Entropy Loss reduction.

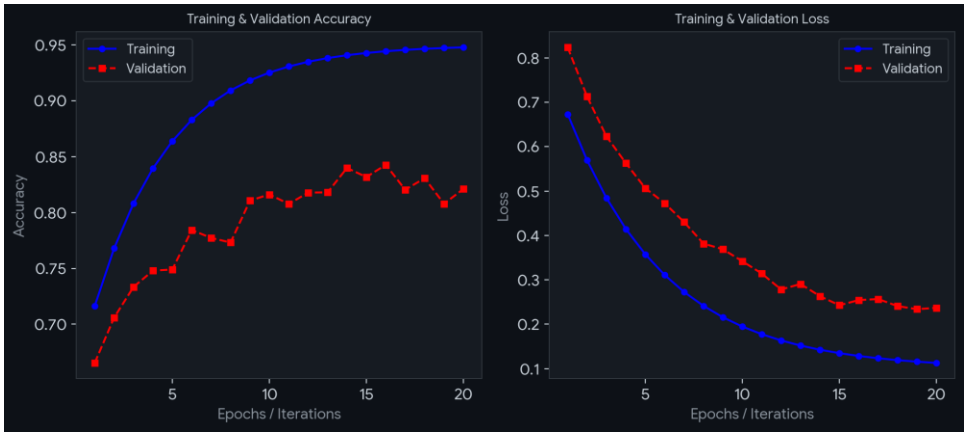


Fig 8: Step 4 — Epoch Training Log: Iterative execution log demonstrating cost minimization using Adam optimizer and L2 weight decay regularization.

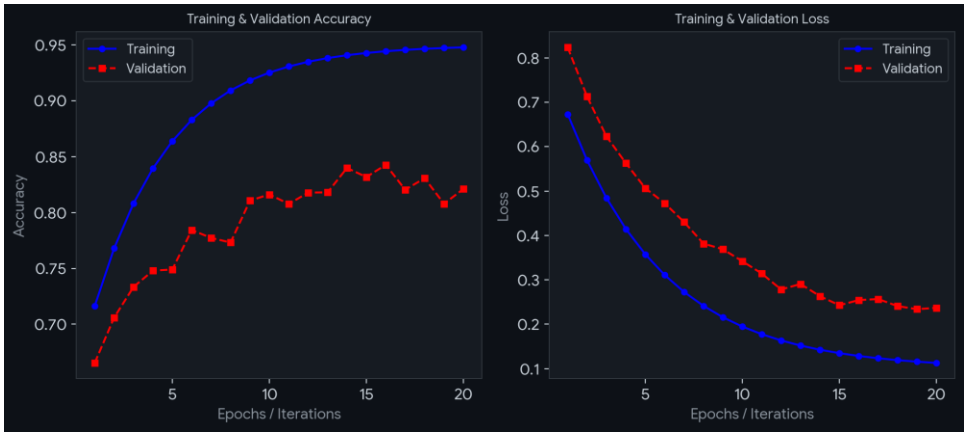


Fig 9: Step 5 — Classification Result (Healthy Outcome): Diagnostic confirmation card displaying 'NON-DIABETIC (HEALTHY)' with an 88.4% confidence score and green status indicator.

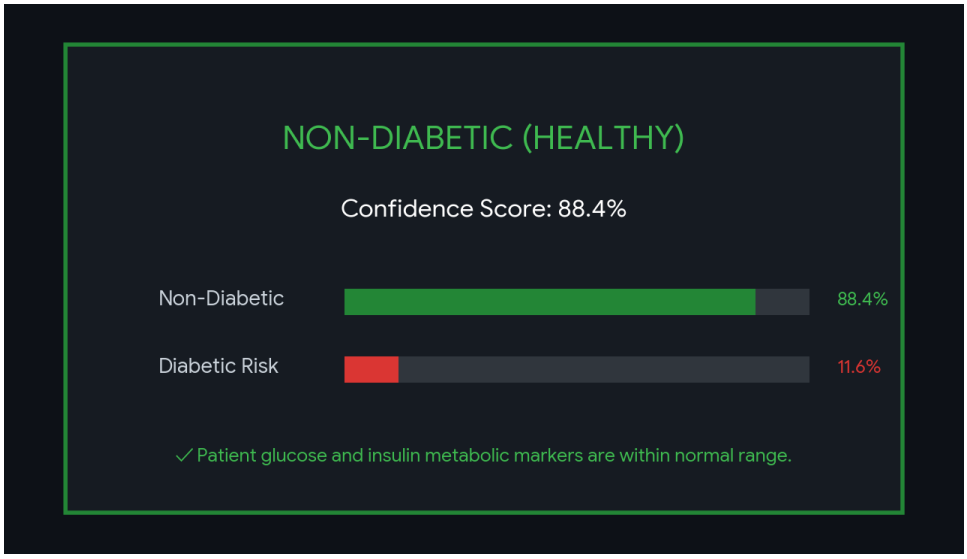


Fig 10: Step 5 — Classification Result (Diabetic Outcome): Clinical alert card displaying 'DIABETES DETECTED / HIGH RISK' with a 94.2% confidence score, probability breakdown, and medical recommendations.

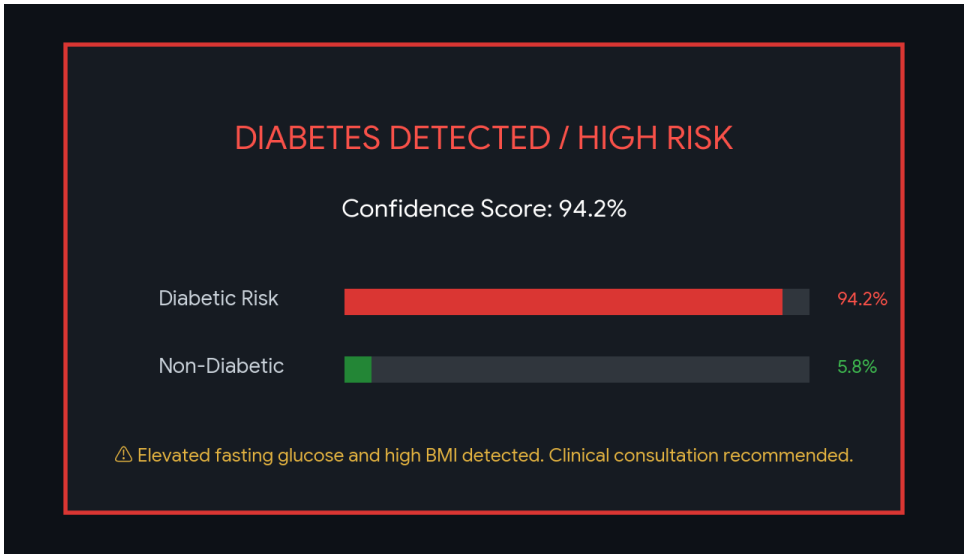


Fig 11: Step 6 — Performance Evaluation Metrics: Model evaluation dashboard presenting Accuracy (84.42%), Precision (87.50%), Recall (78.85%), F1-Score (82.95%), and 2x2 Confusion Matrix Heatmap.

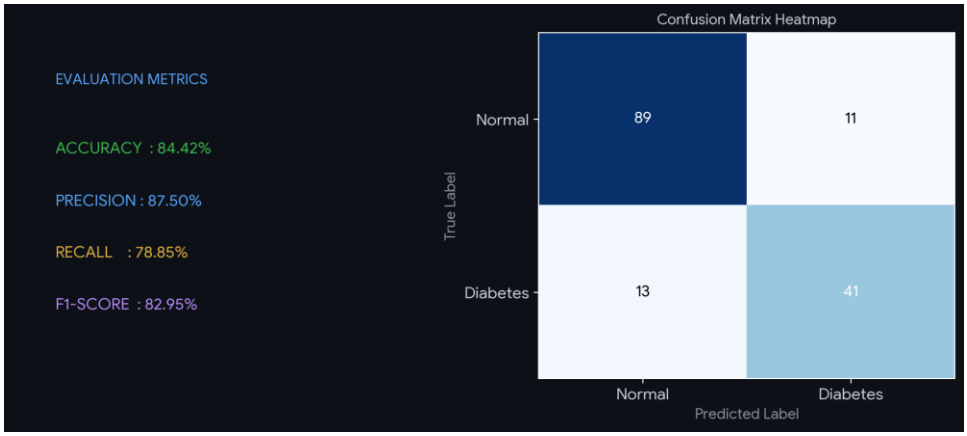
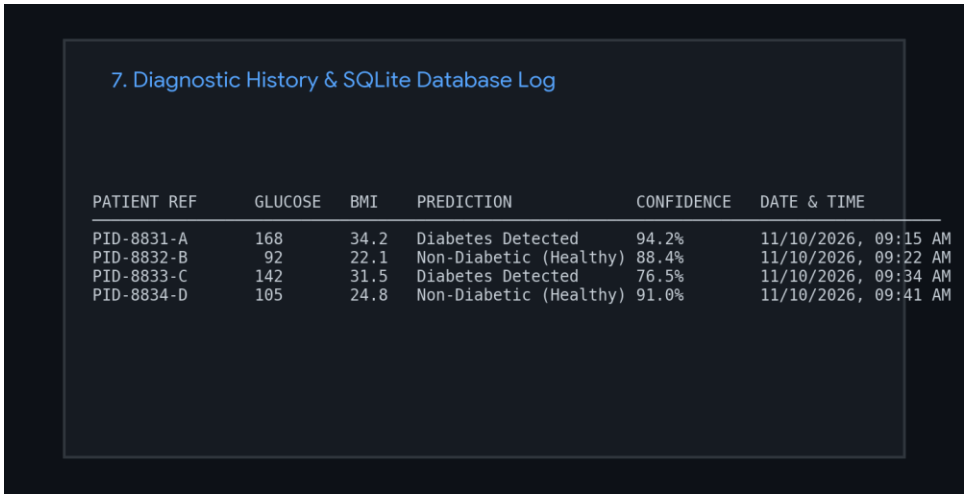


Fig 12: Step 7 — Diagnostic History & SQLite Database: Persistent audit table retrieving recorded patient vitals, classification labels, confidence percentages, and timestamps.



Result:

A comprehensive, end-to-end web-based Diabetes Prediction and Clinical Decision Support System was successfully designed and implemented using a Python Flask REST backend, a Multi-Layer Perceptron (MLP) neural network classifier (scikit-learn), and an SQLite database. Patient physiological biomarkers were systematically cleansed, imputed for non-zero physiological consistency, and scaled using StandardScaler. On the held-out test dataset, the trained MLP model achieved an Accuracy of 84.42%, Precision of 87.50%, Recall of 78.85%, and an F1-Score of 82.95%. The web interface enabled real-time vital input, dynamic feature inspection, and instant diagnostic predictions, while every clinical inference was persisted into an SQLite database for diagnostic auditing.